

1. Мои данные

Козенко Дмитрий; группа Б08; студенческий номер st117373.

2. Полное название задачи

Построение множества $\text{EFF}_k(\alpha)$ для КС-грамматики: префиксы длины k (или целиком более короткой терминальной цепочки), получающиеся по правосторонним выводам из α , где *последнее применённое правило не является ε -порождением* (определение 3.7, слайды 79–86 файла eff.pptx).

3. Теоретическое описание

Пусть $G = (V_N, V_T, P, S)$ — контекстно-свободная грамматика, $\alpha \in (V_N \cup V_T)^*$, задан параметр $k \geq 1$.

Рассматривают правосторонние выводы $\alpha \Rightarrow^* w$, где $w \in V_T^*$.

Множество $\text{EFF}_k(\alpha)$: цепочки $u \in V_T^*$, которые либо сами есть w при $|w| < k$, либо префиксы w длины k ; вывод должен завершаться применением правила с *непустой* правой частью (слайд 84 формулирует сбор префиксов длины k аналогично).

4. Теоретическое описание решения и форматы входных данных

(А) Расширенный формат задаёт те же объекты, что и формулировки на занятии: когда во входе в начале хотя бы одной строки (без учёта регистра) встречаются ключевые слова **start**, **nonterminals** или **terminals**, включается запись грамматики с явными списками символов. Несколько директив допускают записать в одну физическую строку, разделив их символом **;**.

Структура полей:

k (число) длина префикса k для множества EFF_k ;

start (НТ) один нетерминал A–Z;

nonterminals (...) несколько одиночных литер НТ через пробел;

terminals (...) терминалы одной литерой каждый через пробел; не использовать заглавные A–Z здесь как терминалы;

alpha (...) хотя бы одна такая строка — запрос цепочки α для вычисления (символы через пробел);

продукции строки **A -> ...**: справа от стрелки символы грамматики через пробел; альтернативы в одном правиле через **|**; пустое окончание строки после **->** или токены **eps/epsilon** в нужной альтернативе задают ε -ветвь.

Дополнительно возможны параметры ограничителя **max_leaf_print**, **max_form_chars**, **max_nodes**. Строгая проверка: символ во всех правилах и в **alpha** входит только в те **nonterminals** и **terminals**, что перечислены директивами.

(Б) **Компактная запись.** Если ни одной из первых трёх директив нет во входном файле, используют упрощённый текст: строка `k...`; далее в произвольном порядке идут либо цепочки `alpha` (нет подстрочного узла `->`), либо продукции. Заглавная латиница тогда понимается как потенциальный нетерминал только когда такой литерал уже встречается в прочитанных правилах.

Алгоритм. Программа воспроизводит дерево правостроенных порождений, на каждом шаге развёртывается самый правый НТ очередной формы до терминала; у лист фиксируется последнее применённое правило для отсеечения финальных ε -ветвей из итога EFF_k . Дополнительно в отчёт о выводе попадают префиксы без фильтра (все терминальные листья).

5. Программа — листинг `main.cpp`

Перед включением в отчёт исходный файл `main.cpp` пропускают через `../common/filter_program_stdout_for_tex.pl` для подготовки литинга ниже и единообразия вывода.

```
#include "../common/io_ru.hpp"
#include "../common/table_ru.hpp"

#include <algorithm>
#include <cctype>
#include <cstring>
#include <cstdint>
#include <string_view>
#include <iostream>
#include <optional>
#include <set>
#include <sstream>
#include <string>

#include <unordered_set>
#include <vector>

namespace {

struct Production {
    int id = 0;
    std::vector<char> lhs;
    std::vector<char> rhs;
};

std::string strip(std::string s) {
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.front()))) s.erase(0, 1);
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.back()))) s.pop_back();
    return s;
}

bool is_skippable_line(const std::string& line) {
    const std::string t = strip(line);
    return t.empty() || (!t.empty() && t[0] == '#');
}

bool nt_char(unsigned char c) { return 'A' <= c && c <= 'Z'; }

std::string form_to_sv(const std::vector<char>& f) {
    std::string s;
    s.reserve(f.size());
    for (unsigned char x : f) s.push_back(static_cast<char>(x));
    return s;
}

std::string lhs_str(const Production& p) {
```

```

    std::string s;
    if (!p.lhs.empty()) s.push_back(p.lhs.front());
    return s;
}

std::string prod_str_compact(const Production& p) {
    std::ostringstream os;
    os << lhs_str(p) << " -> ";
    if (p.rhs.empty())
        os << std::string{"\xCE\xB5"};
    else {
        for (unsigned char c : p.rhs) os << static_cast<char>(c);
    }
    return os.str();
}

std::optional<int> rightmost_nt_index(const std::vector<char>& cur) {
    for (int i = static_cast<int>(cur.size()) - 1; i >= 0; --i) {
        unsigned char x = cur[static_cast<size_t>(i)];
        if (nt_char(x)) return i;
    }
    return {};
}

bool all_terminal(const std::vector<char>& cur) {
    for (unsigned char x : cur) {
        if (nt_char(x)) return false;
    }
    return true;
}

std::string trunc_prefix(const std::string& term, int k) {
    if (k <= 0) return "";
    const int n = static_cast<int>(term.size());
    if (n <= k) return term;
    return term.substr(0, static_cast<size_t>(k));
}

struct Parsed {
    int k = 1;
    int max_leaf_print = 80;
    int max_form_chars = 64;
    int max_nodes = 200000;
    bool extended_input = false;
    char start_symbol = 0;
    std::unordered_set<char> vn_decl;
    std::unordered_set<char> vt_decl;
    std::vector<char> alphabet_nt;
    std::vector<std::vector<char>> alphas_raw;
    std::vector<Production> prods;
};

bool parse_positive_int(const std::string& tok, int& out) {
    std::vector<char> digs;
    for (unsigned char c : tok)
        if (std::isdigit(c)) digs.push_back(static_cast<char>(c));
    if (digs.empty()) return false;
    long v = 0;
    for (unsigned char d : digs) {
        v = v * 10 + (d - '0');
        if (v > 1000000) return false;
    }
    out = static_cast<int>(v);
    return true;
}

void split_by_semicolons(std::vector<std::string>& out, const std::string& raw) {
    std::string x = strip(raw);
    if (x.empty()) return;
    size_t p = 0;
    while (p < x.size()) {
        size_t q = x.find(';', p);
        if (q == std::string::npos) {

```

```

        auto s = strip(x.substr(p));
        if (!s.empty()) out.push_back(s);
        break;
    }
    auto s = strip(x.substr(p, q - p));
    if (!s.empty()) out.push_back(s);
    p = q + 1;
}
}

bool kw_first_equals_ci(std::string_view seg, std::string_view kw) {
    const std::string s = strip(std::string(seg));
    const size_t ps = s.find(' ');
    const std::string first = strip(ps == std::string::npos ? s : s.substr(0, ps));
    if (first.size() != kw.size()) return false;
    for (size_t i = 0; i < first.size(); ++i)
        if (std::tolower(static_cast<unsigned char>(first[i])) != std::tolower(static_cast<unsigned char>(kw[i])))
            return false;
    return true;
}

std::string rest_after_kw(std::string_view seg_sv, std::string_view kw) {
    std::string s = strip(std::string(seg_sv));
    if (!kw_first_equals_ci(s, kw)) return {};
    const size_t ps = s.find(' ');
    if (ps == std::string::npos) return {};
    return strip(s.substr(ps + 1));
}

bool is_production_fragment(const std::string& seg) {
    const auto pos = seg.find("->");
    if (pos == std::string::npos) return false;
    if (pos + 2 > seg.size()) return false;
    return true;
}

bool tokenize_symbols(const std::string& spaced, std::vector<char>& out_syms, std::string& err) {
    std::istringstream is(spaced);
    std::string tok;
    while (is >> tok) {
        if (tok.size() != 1) {
            err =
                "каждый символ алфавита задается отдельным токеном (разделитель пробел) ; строка \"" +
                spaced + "\"";
            return false;
        }
        out_syms.push_back(tok[0]);
    }
    return true;
}

bool iequals_kw_token(std::string t, std::string_view kw) {
    std::transform(t.begin(), t.end(), t.begin(),
        [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
    std::string k(kw.begin(), kw.end());
    std::transform(k.begin(), k.end(), k.begin(),
        [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
    return t == k;
}

bool add_production_lhs_rhs(Parsed& out, char lhs_nt, std::vector<char>&& rhs_vec, std::string& err) {
    Production pr;
    pr.id = static_cast<int>(out.prods.size());
    pr.lhs.assign(1, lhs_nt);
    pr.rhs = std::move(rhs_vec);
    if (out.extended_input) {
        if (!out.vn_decl.count(lhs_nt)) {
            err =
                std::string("нетерминал '" + lhs_nt +
                    "' в левой части не входит в список 'nonterminals' этого входного файла");
            return false;
        }
    }
    for (unsigned char c : pr.rhs) {

```

```

        if (nt_char(c)) {
            if (!out.vn_decl.count(static_cast<char>(c))) {
                err = std::string("в правилах символ НТ '" + static_cast<char>(c) +
                    "' должен быть в 'nonterminals ...'");
                return false;
            }
        } else {
            if (!out.vt_decl.count(static_cast<char>(c))) {
                err = std::string("в правилах терминал '" + static_cast<char>(c) +
                    "' должен быть в 'terminals ...'");
                return false;
            }
        }
    }
    out.prods.push_back(std::move(pr));
    return true;
}

bool symbols_from_rhs_alt_token(std::string alt, std::vector<char>& rhs, std::string& err) {
    alt = strip(alt);
    if (alt.empty() || iequalws_token(alt, "eps") || iequalws_token(alt, "epsilon") ||
        alt == std::string{"\xCE\xB5"}) {
        rhs.clear();
        return true;
    }
    rhs.clear();
    return tokenize_symbols(alt, rhs, err);
}

bool ingest_production_line(Parsed& out, std::string line, std::string& err) {
    auto pos = line.find("->");
    if (pos == std::string::npos || pos + 2 > line.size()) {
        err = "ожидалась продукция '... -> ...'";
        return false;
    }
    std::string left = strip(line.substr(0, pos));
    std::string right_side = strip(line.substr(pos + 2));
    if (left.size() != 1 || !nt_char(static_cast<unsigned char>(left[0]))) {
        err = "Слева от '->' ровно один нетерминал A-Z: строка \"" + line + "\"";
        return false;
    }
    const char lhs = left[0];
    std::vector<std::string> alts;
    {
        size_t cur = 0;
        while (cur <= right_side.size()) {
            const size_t bar = right_side.find('|', cur);
            if (bar == std::string::npos) {
                alts.push_back(strip(right_side.substr(cur)));
                break;
            }
            alts.push_back(strip(right_side.substr(cur, bar - cur)));
            cur = bar + 1;
        }
    }
    for (const std::string& alt : alts) {
        std::vector<char> rhs;
        if (!symbols_from_rhs_alt_token(alt, rhs, err)) return false;
        std::vector<char> rcopy = rhs;
        if (!add_production_lhs_rhs(out, lhs, std::move(rcopy), err)) return false;
    }
    return true;
}

bool load_file(std::istream& in, Parsed& out, std::string& err) {
    std::vector<std::string> L;
    std::string line;
    while (std::getline(in, line)) {
        if (!is_skippable_line(line)) L.push_back(strip(line));
    }
    if (L.empty()) {
        err = "пустой вход";
    }
}

```

```

        return false;
    }
    std::vector<std::string> segments;
    for (const std::string& phys : L) split_by_semicolons(segments, phys);

    bool saw_extended_kw = false;
    for (const std::string& seg : segments) {
        const std::string s0 = strip(seg);
        if (kw_first_equals_ci(s0, "start") || kw_first_equals_ci(s0, "nonterminals") ||
            kw_first_equals_ci(s0, "terminals"))
            saw_extended_kw = true;
    }

    if (saw_extended_kw) out.extended_input = true;

    if (!out.extended_input) {
        size_t p = 0;
        if (segments.empty()) {
            err = "нет сегментов после разбора";
            return false;
        }
        {
            const std::string& ln = segments[p++];
            if (ln.empty() || std::tolower(static_cast<unsigned char>(ln[0])) != 'k') {
                err = "первый фрагмент (или первый до ';' должен быть 'k <целое>";
                return false;
            }
            const size_t dg = ln.find_first_of("0123456789");
            if (dg == std::string::npos || !parse_positive_int(ln.substr(dg), out.k)) {
                err = "'k <целое>': не найдено положительное число";
                return false;
            }
        }
    }
    while (p < segments.size()) {
        const std::string& ln = segments[p];
        if (is_production_fragment(ln)) {
            if (!ingest_production_line(out, ln, err)) return false;
            ++p;
            continue;
        }
        if (ln.rfind("max_leaf_print ", 0) == 0) {
            if (!parse_positive_int(ln.substr(std::strlen("max_leaf_print ")), out.max_leaf_print)) {
                err = "max_leaf_print: целое";
                return false;
            }
        }
        ++p;
        continue;
    }
    if (ln.rfind("max_form_chars ", 0) == 0) {
        if (!parse_positive_int(ln.substr(std::strlen("max_form_chars ")), out.max_form_chars)) {
            err = "max_form_chars: целое";
            return false;
        }
    }
    ++p;
    continue;
    }
    if (ln.rfind("max_nodes ", 0) == 0) {
        if (!parse_positive_int(ln.substr(std::strlen("max_nodes ")), out.max_nodes)) {
            err = "max_nodes: целое";
            return false;
        }
    }
    ++p;
    continue;
    }
    std::vector<char> alpha_chain;
    {
        std::istringstream is(ln);
        std::string tok;
        bool any = false;
        while (is >> tok) {
            any = true;
            if (tok.size() != 1) {
                err = "строка цепочки alpha: каждый символ отдельным токеном: «" + ln + ">";
            }
        }
    }

```

```

        return false;
    }
    alpha_chain.push_back(tok[0]);
}
if (!any) {
    err = "пустой фрагмент в прежнем формате (ожидалась alpha или правило)";
    return false;
}
}
out.alphas_raw.push_back(std::move(alpha_chain));
++p;
}
if (out.prods.empty()) {
    err = "нет продукций 'A -> ...'";
    return false;
}
if (out.alphas_raw.empty()) {
    err = "укажите хотя бы одну строку цепочки alpha (нет '->') либо запись расширенного формата с директивами start";
    return false;
}
std::unordered_set<char> vn;
for (const Production& pr : out.prods) {
    vn.insert(pr.lhs.front());
    for (unsigned char c : pr.rhs) {
        if (nt_char(c)) vn.insert(static_cast<char>(c));
    }
}
for (char c : vn) out.alphabet_nt.push_back(c);
std::sort(out.alphabet_nt.begin(), out.alphabet_nt.end());
return true;
}

bool have_k = false, have_start = false, have_vn = false, have_vt = false;
for (const std::string& seg0 : segments) {
    const std::string seg = strip(seg0);
    if (seg.empty()) continue;
    if (kw_first_equals_ci(seg, "k")) {
        std::string rest = strip(rest_after_kw(seg, "k"));
        if (rest.empty()) {
            err = "после 'к' должно быть одно число";
            return false;
        }
        std::istringstream ks(rest);
        long v = 0;
        if (!(ks >> v) || v < 1 || v > 1'000'000L) {
            err = "после 'к' должно быть целое от 1 до 1000000";
            return false;
        }
        std::string junk;
        if (ks >> junk) {
            err = "после 'к' давайте только одно число, без дополнительного текста";
            return false;
        }
        out.k = static_cast<int>(v);
        have_k = true;
        continue;
    }
    if (kw_first_equals_ci(seg, "start")) {
        std::string rest = rest_after_kw(seg, "start");
        if (rest.size() != 1 || !nt_char(static_cast<unsigned char>(rest[0]))) {
            err = "'start': один нетерминал A-Z";
            return false;
        }
        out.start_symbol = rest[0];
        have_start = true;
        continue;
    }
    if (kw_first_equals_ci(seg, "nonterminals")) {
        std::string rest = rest_after_kw(seg, "nonterminals");
        std::vector<char> vv;
        if (!tokenize_symbols(rest, vv, err)) return false;
        for (unsigned char cc : vv) {
            if (!nt_char(cc)) {

```

```

        err = "‘nonterminals’ перечисляют только A-Z одиночными литерами";
        return false;
    }
    out.vn_decl.insert(static_cast<char>(cc));
}
have_vn = true;
continue;
}
if (kw_first_equals_ci(seg, "terminals")) {
    std::string rest = rest_after_kw(seg, "terminals");
    std::vector<char> vv;
    if (!tokenize_symbols(rest, vv, err)) return false;
    for (char cc : vv) {
        if (nt_char(static_cast<unsigned char>(cc))) {
            err = "‘terminals’ не должны быть заглавными A-Z (зарезервировано под HT)";
            return false;
        }
        out.vt_decl.insert(cc);
    }
    have_vt = true;
    continue;
}
if (kw_first_equals_ci(seg, "alpha")) {
    std::string rest = rest_after_kw(seg, "alpha");
    std::vector<char> chain;
    if (!tokenize_symbols(rest, chain, err)) return false;
    if (chain.empty()) {
        err = "‘alpha’ не должна быть пустой (для пустой цепочки укажите токен eps на будущее - не поддерживается)";
        return false;
    }
    out.alphas_raw.push_back(std::move(chain));
    continue;
}
if (seg.rfind("max_leaf_print ", 0) == 0) {
    if (!parse_positive_int(seg.substr(std::strlen("max_leaf_print ")), out.max_leaf_print)) {
        err = "max_leaf_print: целое";
        return false;
    }
    continue;
}
if (seg.rfind("max_form_chars ", 0) == 0) {
    if (!parse_positive_int(seg.substr(std::strlen("max_form_chars ")), out.max_form_chars)) {
        err = "max_form_chars: целое";
        return false;
    }
    continue;
}
if (seg.rfind("max_nodes ", 0) == 0) {
    if (!parse_positive_int(seg.substr(std::strlen("max_nodes ")), out.max_nodes)) {
        err = "max_nodes: целое";
        return false;
    }
    continue;
}
if (is_production_fragment(seg)) {
    if (!ingest_production_line(out, seg, err)) return false;
    continue;
}
err = "неизвестный фрагмент “ + seg + “ (ожидаются k/start/nonterminals/terminals/alpha или правило с ‘->’);";
return false;
}

if (!have_k) {
    err = "нужна строка ‘k <число>’";
    return false;
}
if (!have_start) {
    err = "нужна строка ‘start <HT>’";
    return false;
}
if (!have_vn) {
    err = "нужна строка ‘nonterminals ...’";
    return false;
}

```



```

    }
    if (!have_vt) {
        err = "нужна строка 'terminals ...'";
        return false;
    }
    if (!out.vn_decl.count(out.start_symbol)) {
        err = "'start' должен входить в объявленные 'nonterminals'";
        return false;
    }
    if (out.alphas_raw.empty()) {
        err = "нужна хотя бы одна строка 'alpha ...'";
        return false;
    }
    if (out.prods.empty()) {
        err = "нет строк productions 'A -> ...'";
        return false;
    }
    for (char c : out.vn_decl) out.alphabet_nt.push_back(c);
    std::sort(out.alphabet_nt.begin(), out.alphabet_nt.end());
    return true;
}

struct LeafRec {
    std::string term;
    int last_prod_id;
    bool last_is_epsilon;
    std::vector<std::vector<char>> snaps;
};

void dfs_rm(const Parsed& cfg, std::vector<char> cur, std::vector<int>& stack_ids,
            std::vector<std::vector<char>>& snaps, size_t& nodes, bool& truncated,
            std::vector<LeafRec>& leaves) {

    ++nodes;
    if (nodes >= static_cast<size_t>(cfg.max_nodes)) {
        truncated = true;
        return;
    }
    const int nf = static_cast<int>(cur.size());
    if (nf > cfg.max_form_chars) {
        truncated = true;
        return;
    }

    if (all_terminal(cur)) {
        std::string w = form_to_sv(cur);
        LeafRec lr;
        lr.term = std::move(w);
        lr.last_prod_id = -1;
        lr.last_is_epsilon = false;
        if (!stack_ids.empty()) {
            lr.last_prod_id = stack_ids.back();
            lr.last_is_epsilon = cfg.prods.at(static_cast<size_t>(lr.last_prod_id)).rhs.empty();
        } else {
            lr.last_is_epsilon = false;
        }
        lr.snaps = snaps;
        leaves.push_back(std::move(lr));
        return;
    }

    auto idx = rightmost_nt_index(cur);
    if (!idx.has_value()) {
        return;
    }
    unsigned char nt = cur[static_cast<size_t>(*idx)];
    for (const Production& pr : cfg.prods) {
        if (pr.lhs.size() != 1 || static_cast<unsigned char>(pr.lhs.front()) != nt) continue;

        auto new_cur = cur;
        new_cur.erase(new_cur.begin() + *idx);

        auto ins_pos = static_cast<long>(*idx);

```

```

        for (char c : pr.rhs) new_cur.insert(new_cur.begin() + ins_pos, c), ++ins_pos;

        stack_ids.push_back(pr.id);
        snaps.push_back(cur);
        dfs_rm(cfg, std::move(new_cur), stack_ids, snaps, nodes, truncated, leaves);
        snaps.pop_back();
        stack_ids.pop_back();
    }
}

void run_one_alpha(std::ostream& os, const Parsed& cfg, const std::vector<char>& alpha0) {
    os << "\n=====\\n";
    os << " Цепочка alpha = \" << form_to_sv(alpha0) << "\", k = \" << cfg.k << "\\n";
    os << "=====\\n";

    if (all_terminal(alpha0)) {
        const std::string w = form_to_sv(alpha0);
        const std::string pref = trunc_prefix(w, cfg.k);
        table_ru::print_table(
            os, "alpha уже терминальная: EFF_k(alpha) = { префикс alpha длины min(|alpha|, k) }",
            {"Префикс"},
            {{pref.empty() ? std::string("epsilon") : pref}});
        return;
    }

    size_t nodes = 0;
    bool truncated = false;
    std::vector<LeafRec> leaves;
    std::vector<int> st;
    std::vector<std::vector<char>> snaps;
    dfs_rm(cfg, alpha0, st, snaps, nodes, truncated, leaves);

    std::set<std::string> eff;
    std::set<std::string> first_all;
    int shown = 0;

    for (const LeafRec& lr : leaves) {
        const std::string pref = trunc_prefix(lr.term, cfg.k);
        first_all.insert(pref);
        if (!lr.last_is_epsilon) eff.insert(pref);
    }

    os << "Всего листьев (терминальные цепочки по правостороннему выводу): " << leaves.size()
        << '\\n';
    os << "Обойдено узлов дерева вывода: " << nodes;
    if (truncated) os << " (достигнут предел max_nodes/max_form_chars - вывод может быть неполным)";
    os << "\\n\\n";

    for (const LeafRec& lr : leaves) {
        if (shown >= cfg.max_leaf_print) break;
        std::ostringstream chain;
        for (size_t i = 0; i < lr.snaps.size(); ++i) {
            if (i) chain << " => ";
            chain << form_to_sv(lr.snaps[i]);
        }
        if (!lr.term.empty())
            chain << " => " << lr.term;
        else
            chain << " => epsilon";
        const std::string pref = trunc_prefix(lr.term, cfg.k);
        std::string lastp = "-";
        if (lr.last_prod_id >= 0)
            lastp = prod_str_compact(cfg.prods.at(static_cast<size_t>(lr.last_prod_id)));
        const char* tag = lr.last_is_epsilon ? "нет (финал epsilon-правило)" : "да";
        os << "-- вывод #\" << (shown + 1) << " -----\\n";
        os << chain.str() << '\\n';
        os << " терминал w = \" << (lr.term.empty() ? std::string("epsilon") : lr.term)
            << "\", префикс k = \" << (pref.empty() ? std::string("epsilon") : pref) << "\\n\\n";
        os << " последнее правило: " << lastp << " | учитывать в EFF_k: " << tag << '\\n';
        ++shown;
    }

    if (static_cast<int>(leaves.size()) > cfg.max_leaf_print) {
        os << "... показаны первые " << cfg.max_leaf_print << " выводов из " << leaves.size()

```

```

        << " (см. max_leaf_print в входе)\n";
    }
    os << '\n';

    auto set_to_rows = [](const std::set<std::string>& s) {
        std::vector<std::vector<std::string>> rows;
        for (const auto& x : s) rows.push_back({x.empty() ? std::string("epsilon") : x});
        return rows;
    };

    table_ru::print_table(os,
        "Итор: EFF_k -- префиксы, финальное правило не epsilon",
        {"префикс из V_T* длины не больше k"}, set_to_rows(eff));

    table_ru::print_table(
        os,
        "Сравнение: все префиксы с правосторонних листьев (включая финал epsilon-правило)",
        {"префикс"}, set_to_rows(first_all));
}

}

int main(int argc, char** argv) {
    std::ifstream fin;
    if (!io_ru::open_only_read_strict(argc, argv, fin, std::cerr)) return 2;

    std::string err;
    Parsed cfg;
    if (!load_file(fin, cfg, err)) {
        std::cerr << "Ошибка формата входа: " << err << '\n';
        return 1;
    }

    std::cout << "\n----- KC-грамматика (продукции) ----- \n\n";
    {
        std::vector<std::vector<std::string>> rows;
        for (const Production& pr : cfg.prods) {
            rows.push_back({std::to_string(pr.id), lhs_str(pr), prod_str_compact(pr)});
        }
        table_ru::print_table(std::cout, "Продукции P в порядке появления во входном файле",
            {"P", "нетерминал", "правило целиком"}, rows);
        std::vector<std::vector<std::string>> VN;
        for (unsigned char c : cfg.alphabet_nt) VN.push_back({std::string(1, static_cast<char>(c))});
        table_ru::print_table(std::cout,
            "Нетерминалы (при директивном формате --- как в строке nonterminals)", {"VN"},
            VN);
        std::cout << "Число k (определение 3.7): " << cfg.k << '\n';
        if (cfg.extended_input) {
            std::vector<std::vector<std::string>> VT_rows;
            std::vector<char> vt_ord(cfg.vt_decl.begin(), cfg.vt_decl.end());
            std::sort(vt_ord.begin(), vt_ord.end());
            for (char c : vt_ord) VT_rows.push_back({std::string(1, c)});
            table_ru::print_table(std::cout,
                "Терминалы из строки terminals",
                {"терминал"}, VT_rows);
            table_ru::print_table(std::cout, "Стартовый нетерминал из строки start", {"START"},
                {{std::string(1, cfg.start_symbol)}});
        }
    }

    std::unordered_set<char> vnset(cfg.alphabet_nt.begin(), cfg.alphabet_nt.end());

    size_t qi = 0;
    for (const auto& raw : cfg.alphas_raw) {
        for (unsigned char c : raw) {
            if (cfg.extended_input) {
                if (nt_char(c)) {
                    if (!vnset.count(static_cast<char>(c))) {
                        std::cerr << "В цепочке alpha P" << (qi + 1) << " нетерминал '"
                            << static_cast<char>(c) << "' не из 'nonterminals'.\n";
                        return 1;
                    }
                }
            } else if (!cfg.vt_decl.count(static_cast<char>(c))) {

```

```

        std::cerr << "В цепочке alpha №" << (qi + 1) << " терминал '"
        << static_cast<char>(c) << "' не из 'terminals'.\n";
        return 1;
    }
} else {
    unsigned char uc = static_cast<unsigned char>(c);
    bool ok = vnset.count(static_cast<char>(c)) || !nt_char(uc);
    if (!ok) {
        std::cerr << "В запросной цепочке №" << (qi + 1) << " нетерминал '"
        << static_cast<char>(c) << "' не встречается в правилах.\n";
        return 1;
    }
}
}
run_one_alpha(std::cout, cfg, raw);
++qi;
}

std::cout << "\n----- Конец ----- \n\n";
return 0;
}

```

6. Короткая навигация по реализации

kw_first_equals_ci и rest_after_kw распознают директивы расширенного входа (A).

ingest_production_line — альтернативы с |.

dfs_rm — рекурсия ПР разбора под max_nodes/max_form_chars.

7. Файл входных данных sample.txt

Приведён образец входа в расширенной записи (A) примерной грамматикой (3.8 слайдов) с альтернативами через |.

```

k 2
start S
nonterminals S A B C
terminals a b c
alpha S

S -> A B
A -> B a | epsilon
B -> C b | C
C -> c | epsilon

```

8. Пример работы программы на sample.txt

```

----- КС-грамматика (продукции) -----

=====
Продукции P в порядке появления во входном файле
=====
+---+-----+-----+
| № | нетерминал | правило целиком |
+---+-----+-----+
| 0 |          S |          S -> AB |
| 1 |          A |          A -> Ba |
| 2 |          A |          A -> epsilon |
| 3 |          B |          B -> Cb |
| 4 |          B |          B -> C |

```

```

| 5 |          C |          C -> c |
| 6 |          C |          C -> epsilon |
+---+-----+-----+

=====
Нетерминалы (при директивном формате --- как в строке nonterminals)
=====
+---+
| VN |
+---+
| A |
| B |
| C |
| S |
+---+
Число k (определение 3.7): 2

=====
Терминалы из строки terminals
=====
+-----+
| терминал |
+-----+
|      a |
|      b |
|      c |
+-----+

=====
Стартовый нетерминал из строки start
=====
+-----+
| START |
+-----+
|      S |
+-----+

=====
Цепочка alpha = "S", k = 2
=====
Всего листьев (терминальные цепочки по правостороннему выводу): 20
Обойдено узлов дерева вывода: 40

-- вывод #1 -----
S => AB => ACb => Acb => Bacb => Cbacb => cbacb
  терминал w = "cbacb", префикс k = "cb"
  последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #2 -----
S => AB => ACb => Acb => Bacb => Cbacb => bacb
  терминал w = "bacb", префикс k = "ba"
  последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #3 -----
S => AB => ACb => Acb => Bacb => Cacb => cacb
  терминал w = "cacb", префикс k = "ca"
  последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #4 -----
S => AB => ACb => Acb => Bacb => Cacb => acb
  терминал w = "acb", префикс k = "ac"
  последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #5 -----
S => AB => ACb => Acb => cb
  терминал w = "cb", префикс k = "cb"
  последнее правило: A -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #6 -----
S => AB => ACb => Ab => Bab => Cbab => cbab
  терминал w = "cbab", префикс k = "cb"
  последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #7 -----
S => AB => ACb => Ab => Bab => Cbab => bab
  терминал w = "bab", префикс k = "ba"
  последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #8 -----
S => AB => ACb => Ab => Bab => Cab => cab

```

```

    терминал w = "cab", префикс k = "ca"
    последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #9 -----
S => AB => ACb => Ab => Bab => Cab => ab
    терминал w = "ab", префикс k = "ab"
    последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #10 -----
S => AB => ACb => Ab => b
    терминал w = "b", префикс k = "b"
    последнее правило: A -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #11 -----
S => AB => AC => Ac => Bac => Cbac => cbac
    терминал w = "cbac", префикс k = "cb"
    последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #12 -----
S => AB => AC => Ac => Bac => Cbac => bac
    терминал w = "bac", префикс k = "ba"
    последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #13 -----
S => AB => AC => Ac => Bac => Cac => cac
    терминал w = "cac", префикс k = "ca"
    последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #14 -----
S => AB => AC => Ac => Bac => Cac => ac
    терминал w = "ac", префикс k = "ac"
    последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #15 -----
S => AB => AC => Ac => c
    терминал w = "c", префикс k = "c"
    последнее правило: A -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #16 -----
S => AB => AC => A => Ba => Cba => cba
    терминал w = "cba", префикс k = "cb"
    последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #17 -----
S => AB => AC => A => Ba => Cba => ba
    терминал w = "ba", префикс k = "ba"
    последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #18 -----
S => AB => AC => A => Ba => Ca => ca
    терминал w = "ca", префикс k = "ca"
    последнее правило: C -> c | учитывать в EFF_k: да
-- вывод #19 -----
S => AB => AC => A => Ba => Ca => a
    терминал w = "a", префикс k = "a"
    последнее правило: C -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)
-- вывод #20 -----
S => AB => AC => A => epsilon
    терминал w = "epsilon", префикс k = "epsilon"
    последнее правило: A -> epsilon | учитывать в EFF_k: нет (финал epsilon-правило)

```

```

=====
Итог: EFF_k -- префиксы, финальное правило не epsilon
=====

```

```

+-----+
| префикс из V_T* длины не больше k |
+-----+
|                                     ca |
|                                     cb |
+-----+

```

```

=====
Сравнение: все префиксы с правосторонних листьев (включая финал epsilon-правило)
=====

```

```

+-----+
| префикс |
+-----+
| epsilon |
|   a     |
|   ab    |
|   ac    |
|   b     |

```

	ba	
	c	
	ca	
	cb	
+-----+		
-----	Конец	-----

Пояснения к выводу. В начале — таблица продукций, сводка VN; при записи (A) дополнительно печатаются **terminals** и **start**. По строке **alpha S** перечисляются правосторонние листья; завершённые ε -правилом исключаются из итога EFF_k .

9. Ссылка на исходный код

Исходный код задачи доступен по ссылке:

<https://github.com/mitya-y/formal-theories-tp-3-course/tree/master/EFF>